



Spring JMS

1

Spring JMS

2

- ✓ Introduction
- ✓ Architecture autour de JMS
- ✓ Producer / Consumer
- ✓ JMS synchrone / asynchrone
- ✓ JMS par annotations

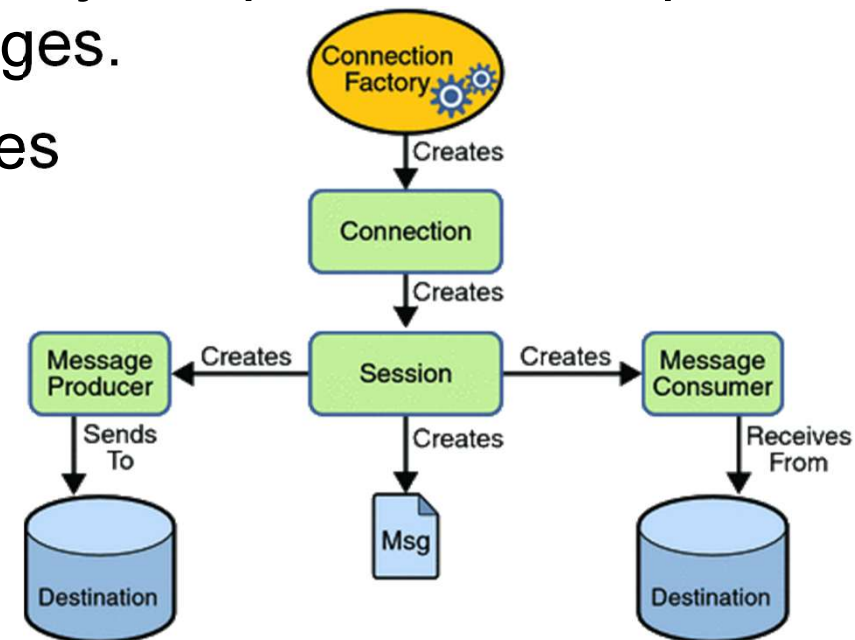
Présentation

- JMS : Java Messaging service
- Spring JMS est un module du Spring
- Il permet de simplifier la mise en place de JMS dans un projet Java
- Spring JMS ne remplace pas un serveur JMS
 - ➡ Il faut toujours en avoir un pour faire du JMS

Rappel JMS

➤ JMS définit plusieurs entités :

- Un **provider** JMS : outil qui implémente l'API JMS pour échanger les messages : ce sont les brokers de messages
- Un **client** JMS : composant écrit en java qui utilise JMS pour émettre et/ou recevoir des messages.
- Un **message** : données échangées entre les composants



JMS

5

- Différents objets utilisés via JMS sont généralement stockés dans l'annuaire JNDI du serveur d'applications ou du provider du MOM (Message-Oriented Middleware):
 - La fabrique de connexions (ConnectionFactory)
 - Les destinations à utiliser (Queue et Topic)
- JMS définit deux modes pour la diffusion des messages :
 - Point à point (Point to point)
 - Publication / souscription (publish/subscribe)

JMS

6

- **Point à point** (Point to point) : un message est envoyé par un producteur et est reçu par un unique consommateur.
 - Le support utilisé pour la mise en œuvre de ce mode est la queue (**queue**).
 - Le message émis est stocké dans la file jusqu'à ce que le consommateur le lise et envoie une notification de réception du message.
 - A ce moment là le message est **supprimé** de la file.
 - Le message peut avoir une date d'expiration.
- **Publication / souscription** (publish/subscribe) : dans ce mode un message est envoyé par un producteur et est reçu par un ou plusieurs consommateurs.
 - Le support utilisé pour la mise en œuvre de ce mode est le sujet (**topic**).
 - Chaque consommateur doit s'abonner à un sujet (souscription).
 - Seuls les messages émis à partir de cet abonnement sont accessibles par le consommateur.

JMS

7

- Les messages sont asynchrones mais JMS définit deux modes pour consommer un message :
 - **Mode synchrone** : ce mode nécessite l'appel de la méthode `receive()` ou d'une de ses surcharges.
 - ❑ Dans ce cas, l'application est arrêtée jusqu'à l'arrivée du message.
 - **Mode asynchrone** : il faut définir un listener qui va lancer un thread attendant les messages et exécutant une méthode dès leur arrivée.
 - ❑ C'est un mode par gestion d'évènement.

JMS - Implémentation

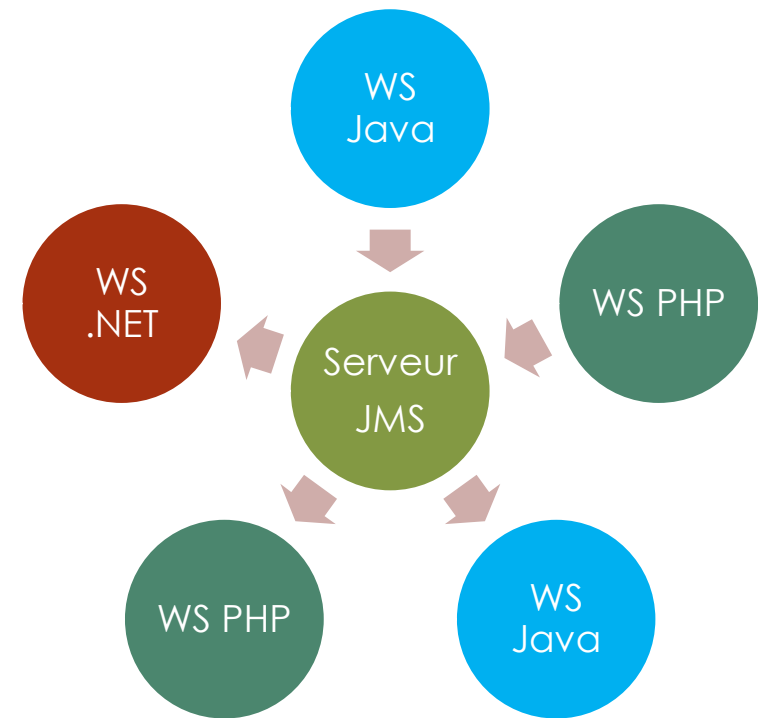
- JMS étant une norme, il faut choisir une implémentation en Java :
 - Apache ActiveMQ
 - OpenJMS
 - JBoss Messaging et HornetQ de JBoss
 - JORAM, de ObjectWeb maintenant OW2
- Avantage d'être une norme, vous trouverez des implémentations en PHP, .NET, Python,

JMS - Implémentation

- Les queues JMS sont très largement utilisées dans les architectures SOA (service Oriented Architecture) afin de permettre aux différentes briques applicatives du SI de communiquer entre elles.
- Normalisation sur la communication entre applications
- Pas d'adhérence directe ou de dépendance technique forte
- Mais attention
 - ➡ Pas toujours simple à architecturer

JMS – Architecture classique

- Le serveur JMS est au centre de tout, il réceptionne toutes les informations qui peuvent servir à d'autres programmes
- C'est comme une base de données, mais en plus simple



Spring et JMS : JmsTemplate

- On va avoir un objet pour simplifier la gestion JMS
 - `org.springframework.jms.core.JmsTemplate`
- Il permet d'envoyer et recevoir des messages
 - `send(...)` : pour envoyer
 - `receive()` : pour recevoir
- Afin de fonctionner correctement cet objet doit être associé à une `javax.jms.ConnectionFactory`
 - `org.apache.activemq.ActiveMQConnectionFactory` : si l'on fait du activeMQ

Spring et JMS : JmsTemplate

➤ Le JmsTemplate doit aussi indiquer la `javax.jms.Destination` choisie

➤ Queue ou Topic

- ☐ `org.apache.activemq.command.ActiveMQQueue` : si l'on cible une queue de activeMQ
- ☐ `org.apache.activemq.command.ActiveMQTopic` : si l'on cible un topic de activeMQ

JMS : Producteur

- Bean Java qui va envoyer des messages
- Sera lié au JmsTemplate

```
import java.util.Date;

import javax.jms.JMSEException;
import javax.jms.Message;
import javax.jms.Session;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jms.core.JmsTemplate;
import org.springframework.jms.core.MessageCreator;
import org.springframework.stereotype.Component;

@Component
public class JmsProducer {

    @Autowired
    private JmsTemplate jmsTemplate;

    public JmsProducer() {
        super();
    }

    public void sendMessage() {
        this.jmsTemplate.send(new MessageCreator() {
            @Override
            public Message createMessage(final Session session)
                throws JMSEException {
                return session.createTextMessage("Send ..." + new Date());
            }
        });
    }
}
```

JMS : Consommateur synchrone

- Bean Java qui va recevoir des messages de manière synchrone
- Sera aussi lié au JmsTemplate

```
import javax.jms.Message;
import javax.jms.TextMessage;

import org.apache.commons.logging.Log;
import org.apache.commons.logging.LogFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jms.core.JmsTemplate;
import org.springframework.stereotype.Component;

@Component
public class JmsConsumer {
    private final static Log LOG = LogFactory.getLog(JmsConsumer.class);

    @Autowired
    private JmsTemplate jmsTemplate;

    public void receiveMessage() {
        try {
            Message msg = this.jmsTemplate.receive();
            TextMessage textMessage = (TextMessage) msg;
            if (msg != null) {
                JmsConsumer.LOG.info("Message = " + textMessage.getText());
            }
        } catch (Exception e) {
            JmsConsumer.LOG.error("Erreur ", e);
        }
    }
}
```

JMS et configuration

- Dans le fichier de configuration Spring on va retrouver les objets indispensables au fonctionnement de JMS

- **ConnectionFactory**

```
<!-- Connexion a activeMQ -->  
<!-- Ne pas oublier de demarrer le serveur activemq.bat start -->  
<bean id="aMQConnectionFactory" class="org.apache.activemq.ActiveMQConnectionFactory">  
  <property name="brokerURL"  
    value="tcp://localhost:61616?wireFormat.maxInactivityDuration=0" />  
</bean>
```

- **Destination**

```
<!-- Nom de la queue de message -->  
<!-- Elle apparaîtra dans la console web http://localhost:8161/admin/queues.jsp -->  
<bean id="destination" class="org.apache.activemq.command.ActiveMQQueue">  
  <constructor-arg value="local.maqueue" />  
</bean>
```

JMS et configuration

16

- Un cache de connexions (pas obligatoire)

```
<!-- Mise en place d'un cache de connexions pour la performance -->
<bean id="cachedConnectionFactory"
      class="org.springframework.jms.connection.CachingConnectionFactory">
  <property name="targetConnectionFactory" ref="amqConnectionFactory" />
  <property name="sessionCacheSize" value="3" />
</bean>
```

- Un ou plusieurs JmsTemplate

```
<!-- JmsTemplate pour le producteur de messages -->
<bean id="producerTemplate" class="org.springframework.jms.core.JmsTemplate">
  <property name="connectionFactory" ref="cachedConnectionFactory" />
  <property name="defaultDestination" ref="destination" />
</bean>
```

- ☐ Attention, dans le cas des topic, pensez à ajouter :
<property name="pubSubDomain" value="true"/> à votre JmsTemplate

Travaux pratiques



- **Exo 18 - Importez le projet Spring JMS**
 - Notez les dépendances Maven vers :
 - ❑ une implémentation de JMS, ici activeMQ
 - ❑ Slf4J
 - ❑ Et bien sur Spring-jms
- **Créez une classe `com.exo.JmsProducer`**
 - Son rôle sera d'envoyer des messages horodatés ⇔ le message contiendra la date sous forme de chaîne de caractères
- **Créez une classe `com.exo.JmsConsumer`**
 - Son rôle sera d'afficher dans le log le message reçu
- **Créez le fichier de configuration Spring afin d'initialiser vos `JmsTemplate` et vos deux classes (`JmsProducer` et `JmsConsumer`)**
- **Créez deux classes**
 - `MainProducer` : chargera le fichier Spring dans une méthode `main` et utilisera votre classe `JmsProducer` pour produire un message
 - `MainConsumer` : chargera le fichier Spring dans une méthode `main` et utilisera votre classe `JmsConsumer` pour lire un message

JMS : Consommateur Asynchrone

- Bean Java qui va recevoir des messages de manière asynchrone
- Ne sera pas lié au JmsTemplate par vos soins
- Fonctionne comme un listener

```
import javax.jms.Message;
import javax.jms.MessageListener;
import javax.jms.TextMessage;

import org.apache.commons.logging.Log;
import org.apache.commons.logging.LogFactory;
import org.springframework.stereotype.Component;

@Component("JmsConsumer")
public class JmsConsumer implements MessageListener {
    private final static Log LOG = LogFactory.getLog(JmsConsumer.class);

    @Override
    public void onMessage(Message msg) {
        try {
            TextMessage textMessage = (TextMessage) msg;
            if (msg != null) {
                if (JmsConsumer.LOG.isInfoEnabled()) {
                    JmsConsumer.LOG.info("Message = " + textMessage.getText());
                }
            }
        } catch (Exception e) {
            JmsConsumer.LOG.error("Erreur ", e);
        }
    }
}
```

JMS : Consommateur Asynchrone

- Le consommateur asynchrone va implémenter l'interface `javax.jms.MessageListener`
- Il va donc écrire la méthode
 - ➡ `public void onMessage(Message msg)`
- Contrairement au consommateur synchrone il ne devra pas être explicitement appelé.
- C'est le Spring qui va le lancer et il fonctionnera automatiquement.

JMS : Consommateur Asynchrone

- Cependant, il faudra déclarer dans le Spring un nouvel élément :
 - ➡ Le listener-container : conteneur de listener

```
<jms:listener-container container-type="default"
                        connection-factory="cachedConnectionFactory"
                        acknowledge="auto">

    <!-- Association avec mon bean listener 'JmsConsumer' -->
    <jms:listener      destination="local.maqueue"
                      ref="JmsConsumer"
                      method="onMessage" />

</jms:listener-container>
```

- Il sera lié à votre ConnectionFactory et contiendra vos consommateurs.
- Chaque consommateur indiquera le nom de la méthode associée à la réception des messages

Travaux pratiques

- Exo 18 - Faites une copie de projet JMS (pensez à modifier votre artefactId) ou modifiez le directement
- Ajustez votre classe `com.exo.JmsConsumer` afin qu'elle soit Asynchrone
 - Ajoutez l'implémentation de l'interface
- Modifiez le fichier de configuration Spring afin qu'il porte le conteneur de listener (faites usage du name space `jms`)
- Créez une nouvelle classe `com.exo.Main` qui, dans une méthode `main`,
 - chargera votre fichier Spring
 - Utilisera le `JmsProducer` afin de fabriquer/envoyer plusieurs messages décalés dans le temps (utilisez `Thread.sleep`)



JMS : Annotations, Spring >= 4

- Dans le cas de l'utilisation de listener, il est possible d'utiliser des annotations (possible à partir de Spring 4).
- L'annotation se place sur la méthode du bean qui doit recevoir les messages
 - `@JmsListener`
- Il faut la compléter en indiquant
 - La destination (le nom de la destination, pas son id)
 - Le nom du conteneur de listener
- Dans le fichier Spring, ne surtout pas oublier :
 - `<jms:annotation-driven/>`

JMS : Annotations, Spring >= 4

```
import javax.jms.Message;
import javax.jms.TextMessage;

import org.apache.commons.logging.Log;
import org.apache.commons.logging.LogFactory;
import org.springframework.jms.annotation.JmsListener;
import org.springframework.stereotype.Component;

@Component
public class JmsConsumer {
    private final static Log LOG = LogFactory.getLog(JmsConsumer.class);

    @JmsListener(destination = "local.maqueueue",
        containerFactory = "jmsListenerContainerFactory")
    public void receive(Message msg) {
        try {
            TextMessage textMessage = (TextMessage) msg;
            if (msg != null) {
                if (JmsConsumer.LOG.isInfoEnabled()) {
                    JmsConsumer.LOG.info("Message = " + textMessage.getText());
                }
            }
        } catch (Exception e) {
            JmsConsumer.LOG.error("Erreur ", e);
        }
    }
}
```

JMS : Annotations, Spring >= 4

- Le lien entre le conteneur et le listener est géré par l'annotation

```
<jms:listener-container container-type="default"
    connection-factory="cachedConnectionFactory"
    acknowledge="auto"
    factory-id="jmsListenerContainerFactory"/>

<!-- Indique que l'on fait usage des annotations JMS -->
<jms:annotation-driven/>
```

- Notez qu'avec les annotations, un même bean peut gérer la réception de plusieurs queue
 - En effet, cette dernière se place sur une méthode et non une classe

Travaux pratiques , Spring >= 4

- Faites une copie de projet JMS (pensez à modifier votre artefactId) ou modifiez le directement
- Ajustez votre classe `com.exo.JmsConsumer` afin qu'elle utilise l'annotation `@JmsListener`
 - ➡ N'oubliez pas de retirer l'interface
- Modifiez le fichier de configuration Spring afin qu'il pense à faire usage de votre annotation
- Relancez votre Main



JMS : Pour aller plus loin

- Votre ConnectionFactory ainsi que votre conteneur de listeners sont les deux parties critiques de JMS
- Chacun d'eux peut se configurer de manière très précise
 - Comme on le ferait sur un pool de connexion (une datasource)
- Pensez à consulter leur documentation
 - <http://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/jms/listener/DefaultMessageListenerContainer.html>
 - <http://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/jms/connection/CachingConnectionFactory.html>

JMS : Pour aller plus loin

➤ Documentation Spring officielle

- <http://docs.spring.io/spring-framework/docs/current/spring-framework-reference/html/jms.html>
- <https://spring.io/guides/gs/messaging-jms/>